



University Of Asia Pacific

Course Title: Object-Oriented Programming and Web Programming Lab

Course Code: CSE 302

Team Members:

Tanisha Taranoon Hridy: 22201169

Md. Sajid Shahan Rahman: 22201186

Sanjida Rahman: 22201187

Emon: 22101204

Submitted To:

Nuzhat Tabassum Progga

Lecturer

Department of Computer Science & Engineering

University of Asia Pacific

Contents

Introduction.....	3
1.1 Objective.....	3
1.2 Motivation.....	3
1.3 Critical Challenges.....	3
1.4 Social and Environmental Impact.....	3
Project Description.....	4
2.1 Description of Project.....	4
2.2 Proposed Functionalities/Features.....	4
2.3 User control.....	4
Project Design.....	5
3.1 Architecture of the Proposed System.....	5
Models:.....	5
Views:.....	5
Template:.....	6
3.2 ER Diagram.....	10
Project Implementation.....	11
4.1 Tools.....	11
4.2 Example Walk-through.....	12
Team Contributions.....	24
Conclusion and Future Work.....	25
Conclusion.....	25
Future Work.....	25
CEP Mapping.....	26

Introduction

1.1 Objective

This project provides a web application platform, and users will be able to access Event services and event products online. The objective of this project is to build a two-user-based Event Planning and Management Website that simplifies the process of planning and organizing events. It provides vendors the ability to showcase services or products and allows customers to view and utilize them while managing their events through features like checklists, scheduling, and budgeting.

1.2 Motivation

Planning events can be overwhelming and time-consuming. In Bangladesh and many similar regions, event planning still heavily relies on manual efforts. This platform aims to offer users a simple, centralized online tool for organizing events efficiently, minimizing stress, and improving the experience.

1.3 Critical Challenges

- Designing a flexible system that supports multiple user types.
- Handling secure user authentication and authorization.
- Managing and associating user-specific data (e.g., checklists, schedules).
- Time constraints to develop and test core features

1.4 Social and Environmental Impact

This website promotes digital adoption in event planning, potentially reducing paper waste (due to digital checklists and reminders) and increasing accessibility to vendors. It supports small businesses by providing them with a platform to connect with customers directly.

Project Description

2.1 Description of Project

Our ‘ManageHoise?’ website is a Django-based web application designed to streamline the process of planning and organizing events. It supports two types of users: vendors and customers. Vendors can register on the platform and add, update, or remove products or services they offer for events, such as catering, decoration, or rental items. Customers, on the other hand, can browse these offerings, manage their user profiles, and eventually plan their events using various tools. Although the core planning features, such as checklists, budgeting, scheduling, and reminders, have not yet been implemented, the website has laid the foundation for a structured and expandable system. The site utilizes Django’s model-view-template (MVT) architecture to manage data flow and user interaction. It also includes essential features such as user authentication, profile editing, and role-based access, making it a functional two-user platform. Future developments aim to incorporate interactive planning features that will enhance the user experience and make the platform a comprehensive solution for event management.

2.2 Proposed Functionalities/Features

1. Two-user system (Vendor and Customer)
2. Vendor can add/edit/delete items (services or products)
3. Customer can view and purchase items
4. Profile creation and editing
5. Planned: Checklist creation, Budget tracking, Scheduling, Notifications
6. The customer can see the vendor list and the vendor profile

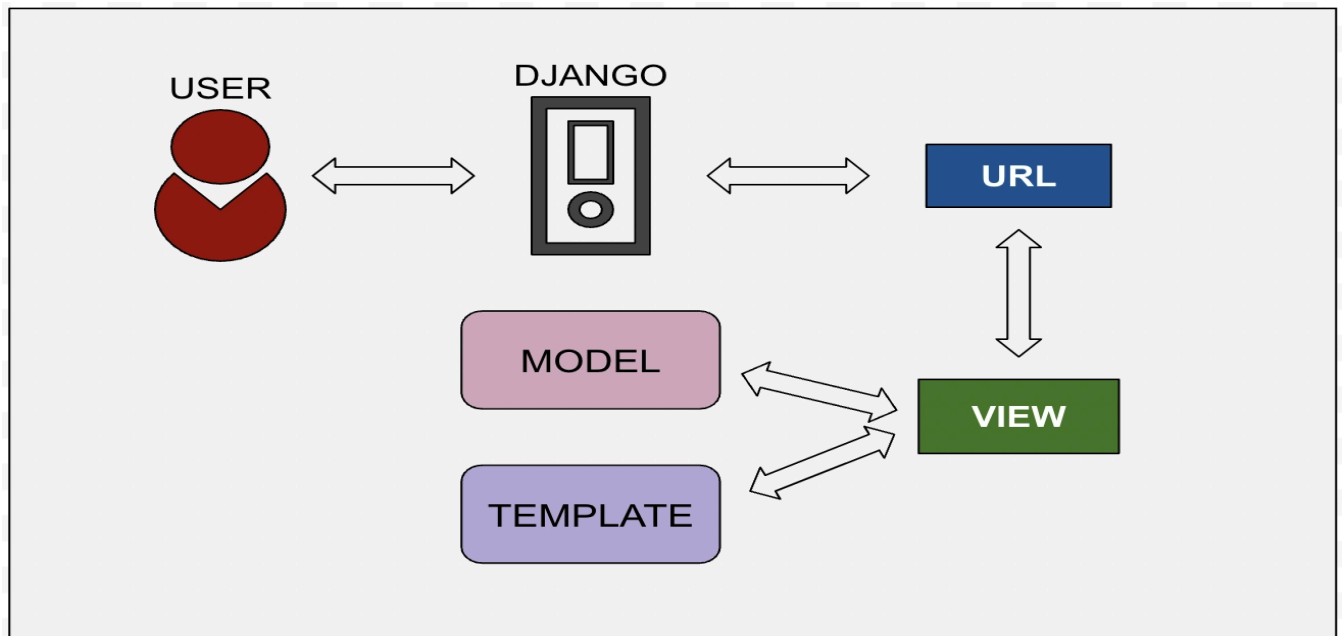
2.3 User control

- Vendors have control over their products.
- Customers can create accounts, manage profiles, and use event tools.
- Admin can manage all users and data through the Django admin panel.

Project Design

3.1 Architecture of the Proposed System

We developed 'ManageHoise?' in Django, which follows the MVT(Model-View-Template) structure. MVT in Django provides a structured approach for separating concerns in web development, promoting code modularity and maintainability. Models handle data, views handle user interactions, and templates handle the presentation, contributing to a scalable and organized project architecture.



Models:

The **models.py** file for the 'ManageHoise?' project encapsulates the data structure for a management system. Key models include C_Profile for user details, Product for product, and Cart Item to manage items in a user's cart. Additionally, an event model for our previous work. Relationships between these models are established to capture the complex interactions within the home domain, providing a solid foundation for data management and retrieval in the 'managehoise?' project.

Views:

The `'views.py'` file handles all the core logic for rendering pages and processing user interactions in our Django Event Planner website. It begins by importing necessary modules, including Django shortcuts like `'render'`, and `'redirect'`, and models and forms that interact with the database. Several views manage product functionality: `'add_product'`, `'update_product'`, and `'delete_product'` allow

vendors to create, modify, and remove their products using a form and conditional logic based on HTTP POST requests. Shopping cart functionality is implemented through views like ``add_to_cart``, ``view_cart``, ``remove_from_cart``, and ``checkout``, enabling authenticated users to manage products in their cart, calculate totals, and place orders. The ``order_history`` view displays a logged-in user's past orders.

The homepage and various static content pages, such as help, policy, and team profiles, are handled by simple ``render`` calls. The ``product`` view dynamically fetches and displays all products and determines the logged-in user's role to personalize the interface. There is also a ``search`` view that enables keyword-based product search. The ``signup`` view processes new user registration using a custom form, validates the uniqueness of username and email, creates a corresponding ``C_profile``, and logs in the user. Similarly, the ``user_login`` and ``user_logout`` views handle authentication using Django's built-in form and functions. The ``profile_view`` allows users to see and update their profile, handling both GET and POST requests and includes logic to check which fields were changed before saving.

Other views like ``vendor_list`` and ``vendor_profile`` allow customers to browse vendor accounts, while event-related views, such as ``event_list`` and ``event_details`` are included to display portfolios. Lastly, various profile views for team members are set up to render personal profile templates. Overall, this file controls the flow of user interactions, connects templates to backend logic, and ensures that only authenticated users can perform certain actions.

Template:

In our Event Management website, Django templates form the backbone of the frontend design, ensuring a modular, consistent, and maintainable user interface. Templates are structured hierarchically to promote reusability and uniform styling across pages. At the core of this structure lies a base template, typically named `base.html`, which contains shared elements like the header, navigation bar, and footer. Individual pages extend this base template and define their content using Django's block syntax.

1. `{% extends %}`:

The `{% extends %}` tag allows child templates (such as `product.html`, `cart.html`, or `profile.html`) to inherit from `base.html`. This ensures that all pages share the same layout, branding, and styling, reducing repetition and promoting consistency throughout the site.

2. `{% block %}`:

The `{% block %}` tag is used within `base.html` to define content areas (e.g., `{% block content %}`) that child templates can override. For example, the `product.html` template might override `{% block content %}` to display product cards, while the `cart.html` page displays items in a user's shopping cart. This modularity simplifies future updates and makes template management more efficient.

This use of template inheritance greatly streamlines development and enables rapid implementation of changes across the entire platform from a single base template.

```
82
83     {% include 'header.html' %}
84     <div class="container">
85         <div class="product-wrapper">
86             <div class="col-md-5 product-image">
87                 {% if product.p_photo %}
88                     
89                 {% else %}
90                     
91                 {% endif %}
92                 <p class="zoom-note">Click to zoom</p>
93             </div>
94
template > <> header.html > header#mainHeader.header > div.header-container > nav.nav-bar
1     {% load static %}
2     <header class="header" id="mainHeader">
3         <div class="header-container">
4             <!-- Left: Logo or Brand Name -->
5             <div class="logo">
6                 <a href="{% url 'home' %}">Follow link (ctrl + click)</a>
7                 
8             </div>
9         </div>
10
```

manage.py:

The manage.py file is a command-line utility that is included in every Django project by default. It acts as the central entry point for executing administrative tasks within the Django environment. This includes operations such as launching the development server using the runserver command, creating new applications, generating and applying database migrations via makemigrations and migrate, and interacting with the Django shell for testing and debugging. Overall, manage.py facilitates the management and execution of core Django functionalities throughout the development lifecycle.

settings.py:

The settings.py file contains the primary configuration for the Django project. It defines various project-wide settings, including database connection parameters, time zone and localization preferences, middleware stack configurations, static and media file directories, and template rendering settings. One of its most critical components is the INSTALLED_APPS list, which registers all Django and custom apps to be recognized and used by the framework. This file serves as the backbone of the project's operational behavior.

admin.py:

The admin.py file allows Django models to be registered with the Django admin interface. By registering

models such as Category, Product, C_profile, Portfolio, CartItem, Order, and OrderItem using `admin.site.register()`, administrative users can manage application data through a graphical interface provided by Django's admin panel. This integration simplifies backend data management and provides a quick way to view, update, and delete records without needing custom dashboards.

forms.py:

The `forms.py` file defines Django form classes that are used for processing user input and validating submitted data. It contains the following key forms:

- **ProductForm:**
 - Type: `ModelForm`
 - Model: `Product`
 - Purpose: Enables vendors to add or update product listings.
 - Usage: Implemented in the `add_product` and `update_product` views.
- **SignUpForm:**
 - Type: `Custom ModelForm`
 - Model: `User` (with extended fields from `C_profile`)
 - Purpose: Manages user registration, including custom fields such as role, phone, profile picture, and bio.
 - Validation: Ensures unique usernames and matching passwords.
 - Effect: Simultaneously creates `User` and corresponding `C_profile` objects.
 - Usage: Used in the `signup` view.
- **ProfilePictureForm:**
 - Type: `ModelForm`
 - Model: `C_profile`
 - Field: `profile_picture`
 - Purpose: Designed to update a user's profile image.
 - Validation: Restricts image size to a maximum of 5 MB.
 - Note: This form has not been integrated into `views.py` but can be used within the profile view to support image updates.

urls.py:

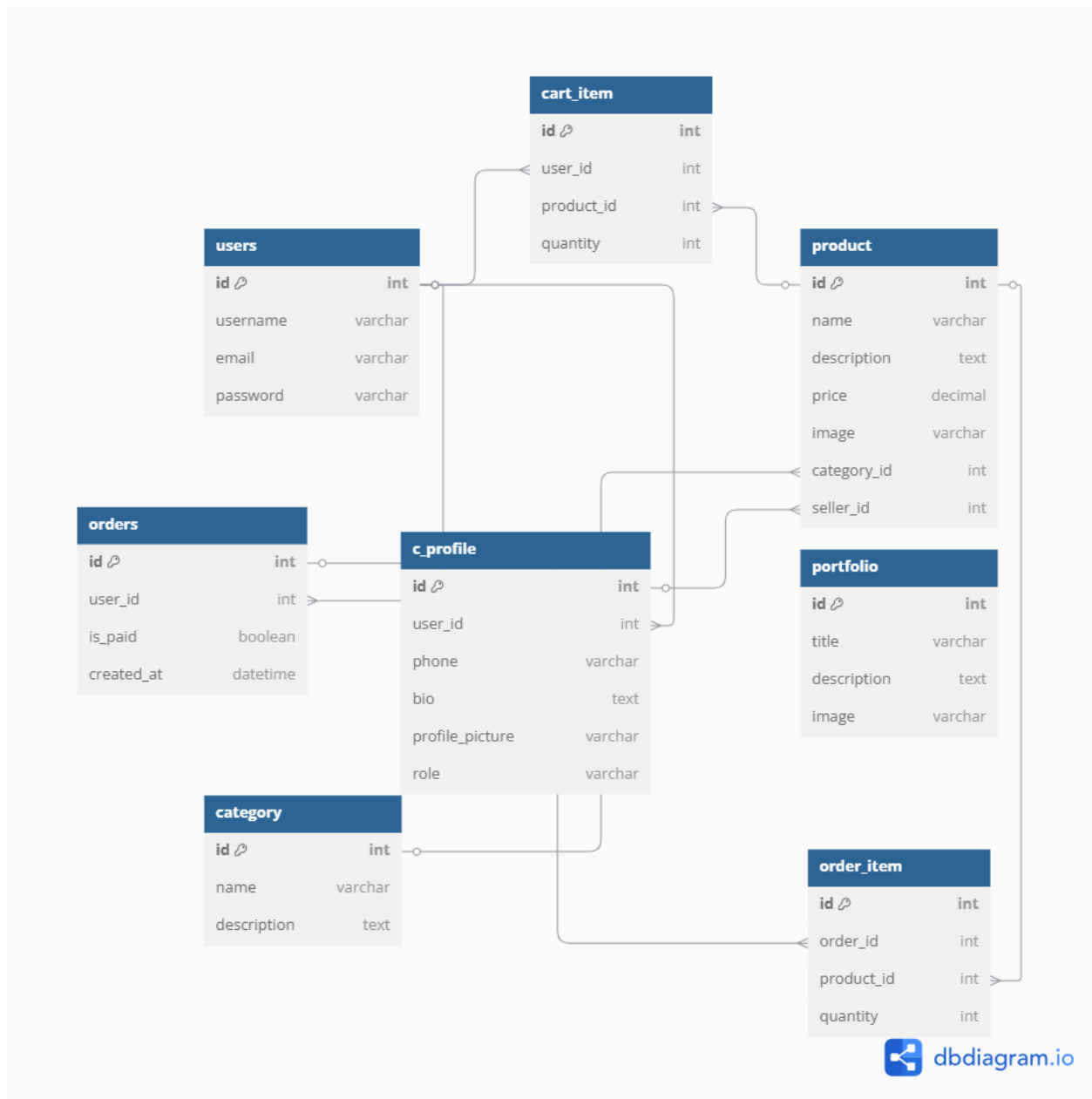
The `urls.py` file contains all the URL pattern definitions that connect frontend URLs to backend view logic. Each path specifies a route and links it to an appropriate view function. The following sections describe its structure:

- **Admin Panel**
 - `admin/`: Provides access to Django's built-in administrative interface.
- **Homepage and Authentication**
 - `/`: Loads the homepage via the `home` view.
 - `/signup/`: Handles user registration via the `signup` view.
 - `/login/`: Directs to the user login page.

- /logout/: Logs out the currently authenticated user.
 - /profile/: Displays and allows updates to user profile details.
- Product Management
 - /product/: Displays a list of all available products.
 - /product/[str:id](#)/: Shows details of a specific product.
 - /product/add_product/: Allows vendors to add a new product.
 - /product/update_product/[str:id](#)/: Enables product updates.
 - /product/delete_product/[str:id](#)/: Facilitates product deletion.
- Team Profiles
 - /meet_the_team/: Shows the general team overview page.
 - Subroutes such as /meet_the_team/sajid/ and /meet_the_team/tanisha/ load individual team member profiles.
- Static Pages
 - /policy/: Loads the privacy policy page.
 - /help/: Loads the help and FAQ section.
- Event Section
 - /event_list/: Displays all portfolio events.
 - /event_list/[int:pk](#)/: Loads detailed information for a specific event.
- Shopping Cart and Orders
 - /cart/: Displays the current user's cart contents.
 - /cart/add/[int:product_id](#)/: Adds a product to the cart.
 - /cart/remove/[int:item_id](#)/: Removes a product from the cart.
 - /checkout/: Finalizes the order and clears the cart.
 - /orders/: Displays the authenticated user's past orders.
- Search and Vendors
 - /search/: Provides keyword search functionality for products.
 - /vendors/: Displays a list of all vendor profiles.
 - /vendor/[int:user_id](#)/: Loads the profile of a specific vendor.
- Media Handling

During development, static and media files (such as uploaded images) are served using the static () function with the MEDIA_URL setting from settings.py. This allows uploaded content to be displayed correctly in templates and user profiles.

3.2 ER Diagram



Project Implementation

4.1 Tools

We used many tools, languages, frameworks, and other things to develop this project and enhance our experience while also maintaining efficiency. The tools are:

VS Code:

In the context of our project, ManageHoise, an online event management website featuring event product buying, renting, and planning events, Visual Studio Code (VS Code) emerges as a preferred development environment. Leveraging VS Code's lightweight design and powerful capabilities enhances the efficiency of our Django-based project. We used its extensions like:

HTML, CSS, Bootstrap:

For the frontend development of our ManageHoise? Project, which focuses on providing comprehensive online event management services, HTML, CSS, and JavaScript play pivotal roles in creating a seamless and user-friendly interface.

HTML serves as the structural foundation, defining the layout and organization of various elements on our web pages.

CSS comes into play for styling, ensuring a visually appealing design and consistent branding across the site. We also use

Bootstrap is a CSS framework to works with many buttons and fonts to style elements.

Python and Django:

In the backend development of our manageHoise? Project, Python is the cornerstone, chosen for its suitability in creating a robust and scalable environment. Python, being a versatile and readable language, is instrumental in crafting the intricate logic that powers the backend functionalities of our online medical platform.

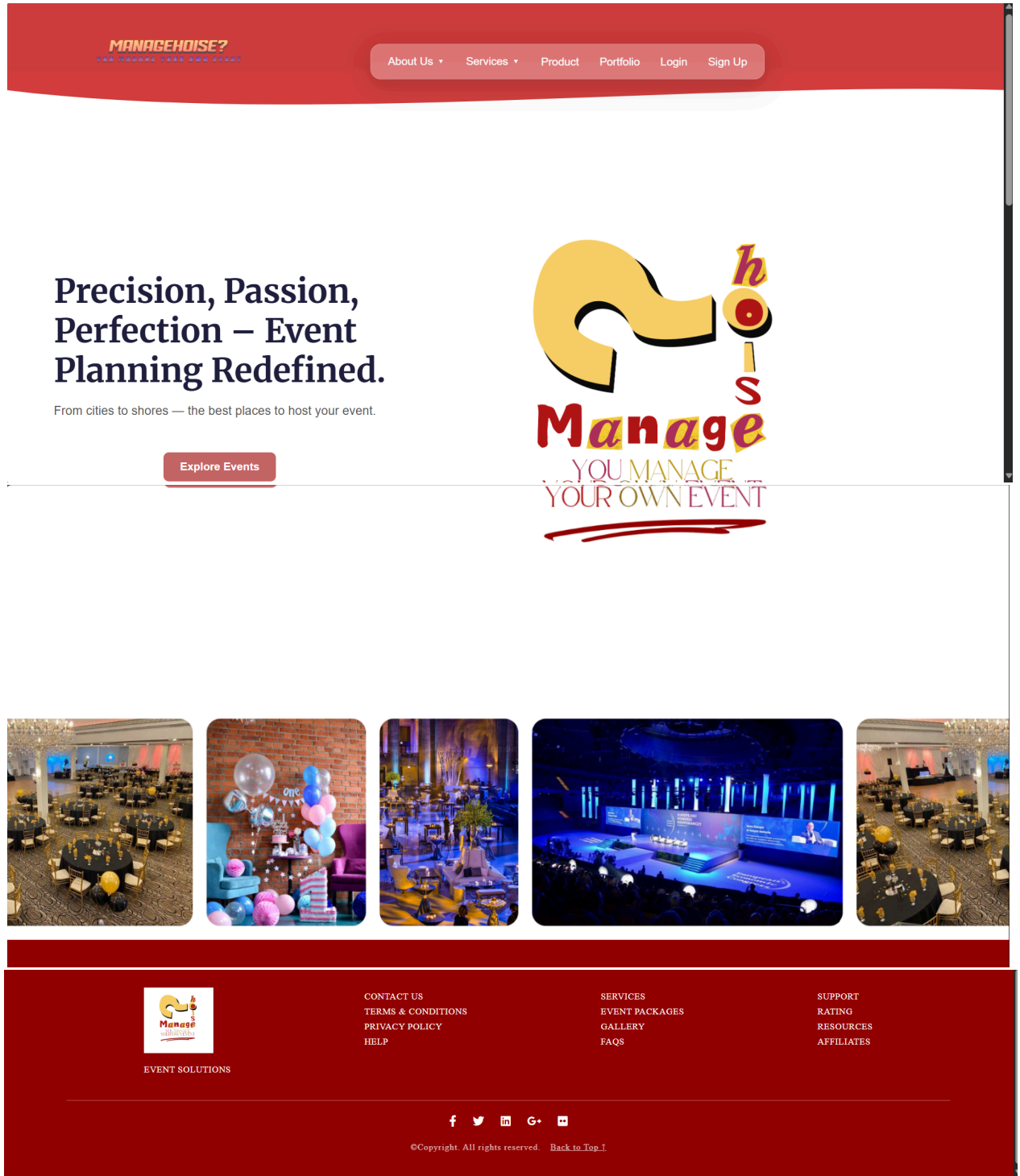
The Django framework, built on Python, provides a structured foundation that streamlines the development process, allowing us to efficiently manage various aspects of our project, such as product searching. Additionally, its extensive ecosystem and Django's built-in features simplify tasks such as database management, ensuring that our backend operations are not only powerful but also maintainable.

Database (SQLite3):

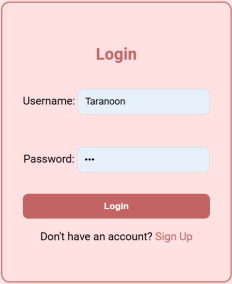
This is the default database of Django, and we used it to store all of the data. We can also update and delete the data from the database. We can use filters and queries to get the desired data from the database.

4.2 Example Walk-through

- Landing Page :

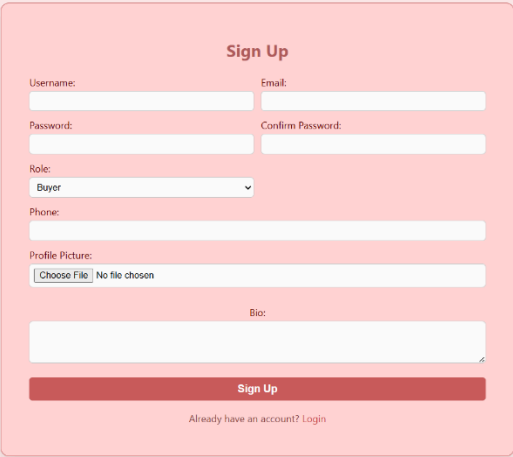


- Login :



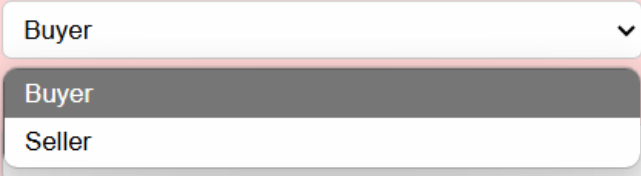
A login form titled "Login" with a light red background. It contains a "Username:" label followed by a text input field containing "Taranoon". Below it is a "Password:" label followed by a password input field with three dots. A red "Login" button is positioned below the password field. At the bottom, there is a link that says "Don't have an account? Sign Up".

- Sign Up



A sign up form titled "Sign Up" with a light red background. It includes fields for "Username:", "Email:", "Password:", and "Confirm Password:". Below these is a "Role:" dropdown menu currently set to "Buyer". There are also fields for "Phone:" and "Profile Picture:" (with a "Choose File" button and "No file chosen" text). A "Bio:" text area is at the bottom. A red "Sign Up" button is centered at the bottom, with a link "Already have an account? Login" below it.

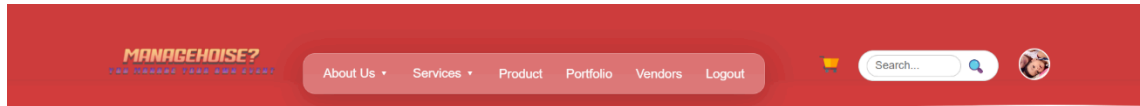
- The role is determined during signup.



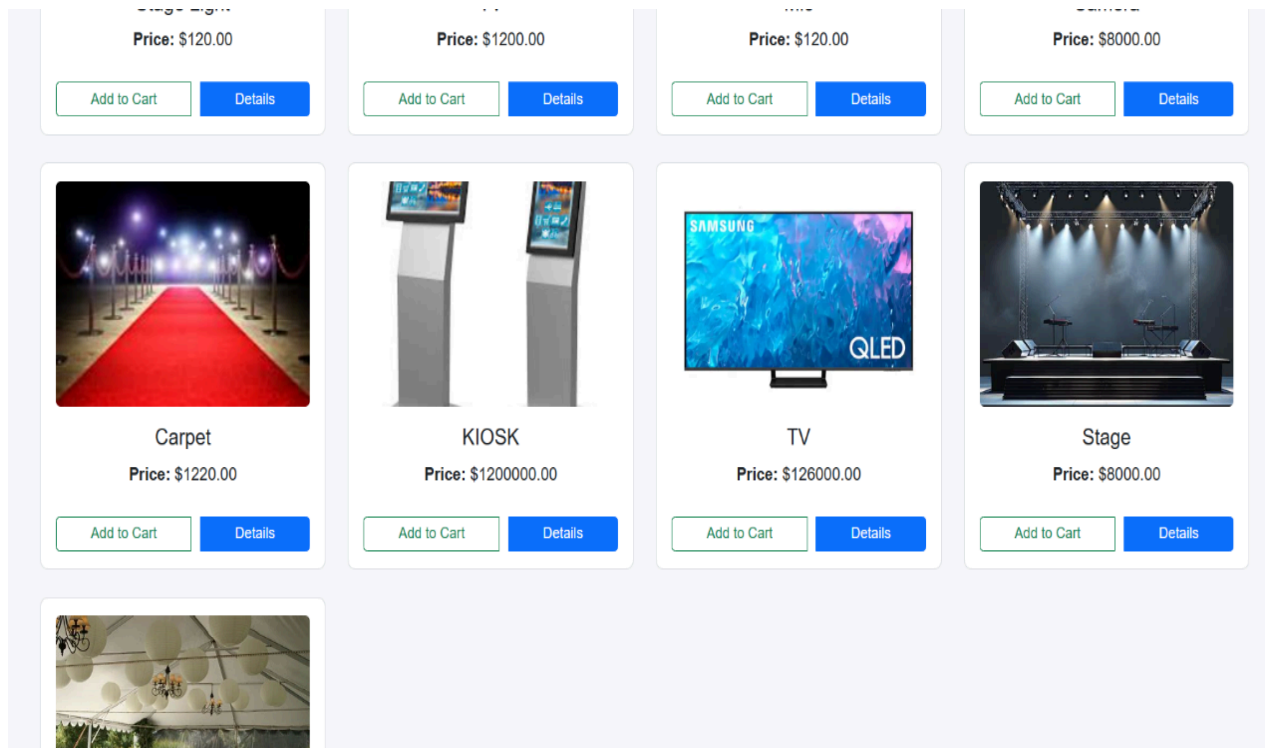
A close-up of the "Role:" dropdown menu. The dropdown is open, showing two options: "Buyer" (highlighted in dark grey) and "Seller". The "Profile Picture:" label is visible below the dropdown.

After logging in :

- As Buyer:



- After Clicking Product:



- Product Details



Click to zoom

TV

₺ **126000.00**

Category: visual product

Status: In Stock

Samsung 65Q70C 65 Inch QLED 4K UHD Smart TV

Added on: May 4, 2025, 3:57 p.m.

Add to Cart

← Back to Products

- Add to cart

Your Cart

	Mic 1 item(s)	\$120.00
	KIOSK 2 item(s)	\$1200000.00
	TV 2 item(s)	\$1200.00
	TV 1 item(s)	\$126000.00

Summary

Subtotal:	\$2528520.00
Total:	\$2528520.00

Proceed to Checkout

- Vendor List

Vendor List



@Sajid

[View Profile](#)



tanisha tara

@Taranoon

[View Profile](#)

- Vendor Profile



tanisha tara

Username: Taranoon

Phone: [REDACTED]

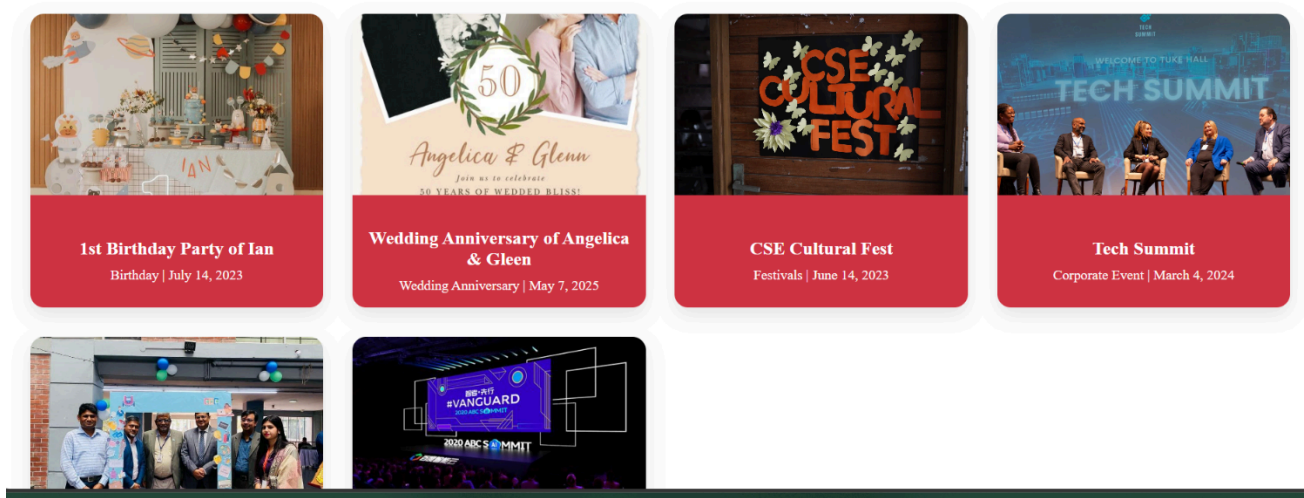
Address: 70/4

Bio:

Role: seller

[Back to Vendor List](#)

- Portfolio



- Portfolio Details



Wedding Anniversary of Angelica & Glenn

Type: Wedding Anniversary
Date: May 7, 2025
Location: Cornelia St, New York, NY 10014, USA
Status: completed
Team/Individual: CakeBoss, ClickMe, Toma

May 7, 2025

[Back](#)

By Admin

- Meet the team

Introducing the people of 🏡



Sanjida Rahman
Chairman



Sajid Shahan Rahman
Managing Director



Tanisha
Chief Executive Officer (CEO)



Sydul Islam Zubair
Event Coordinator



F.H.Shaily
Creative Director



Asif Iqbal
Marketing and Public Relations Specialist



- Team Member Details

MEET OUR TALENTED EVENT COORDINATOR

Sydul Islam Zubair

sydulislam@gmail.com

01516564589

🔗 LinkedIn: @sydulislamzubair

44/i3

West Rajabazar,
Farmgate



- Policy

Privacy Policy:

Last Edited : 1 May 2025

Greetings Users,

This privacy policy describes how the ManageHoise?.com website and related application collects, uses, shares and protects the personal information that we collect through this Site. Please read below to learn more about our information practices. By using this Site, you agree to these practices.

INFORMATION PROVIDED BY YOUR WEB BROWSER

You have to provide us with personal information like your name, contact no, mailing address and email id, our web will also fetch your location information in order to give you the best service. Like many other websites, we may record information that your web browser routinely shares, such as your browser type.

I read the Privacy Notice.

[Yes](#)

- Buyer Profile



Adyaan Safi-u-hossian

Username: adyaan

First Name:

Adyaan

Last Name:

Safi-u-hossian

Phone:

01600876590

Address:

70/4 ,quantum kamal

Bio:

I want to buy things

Role

buyer

You are a **Buyer**. View your orders and cart.

My Orders

Edit Profile

Logout

- Edit profile



Choose File No file chosen

Adyaan Safi-u-hossian

Username: adyaan

First Name:

Adyaan

Last Name:

Safi-u-hossian

Phone:

01600876590

Address:

70/4 ,quantum kamal

Bio:

I want to buy things

Role

buyer

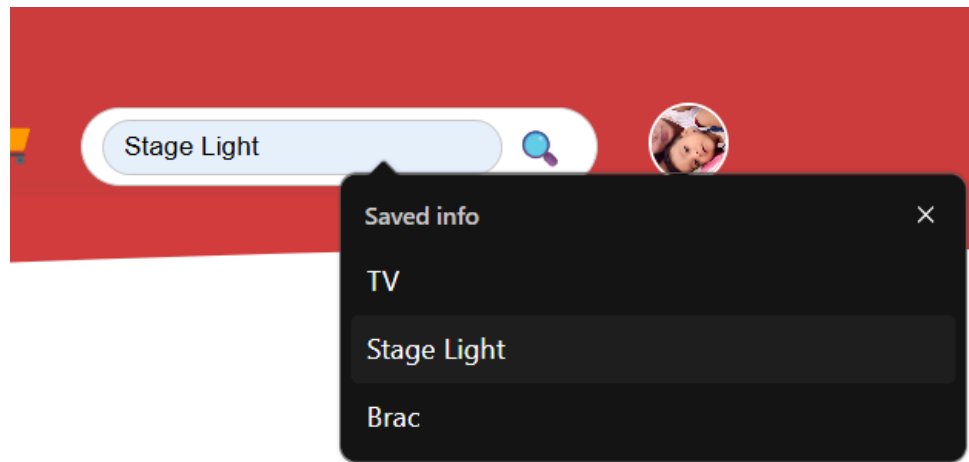
Save Changes

You are a **Buyer**. View your orders and cart.

 My Orders

 Logout

- Search



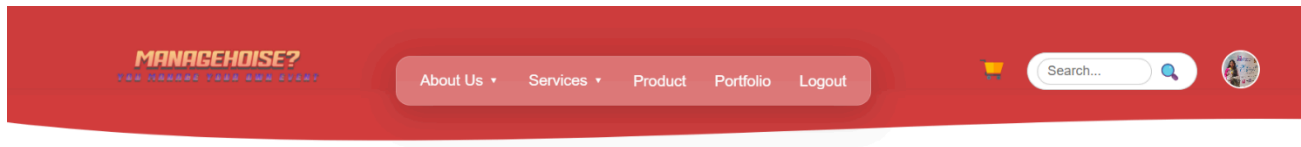
- Search list

Search Results for "Stage Light"

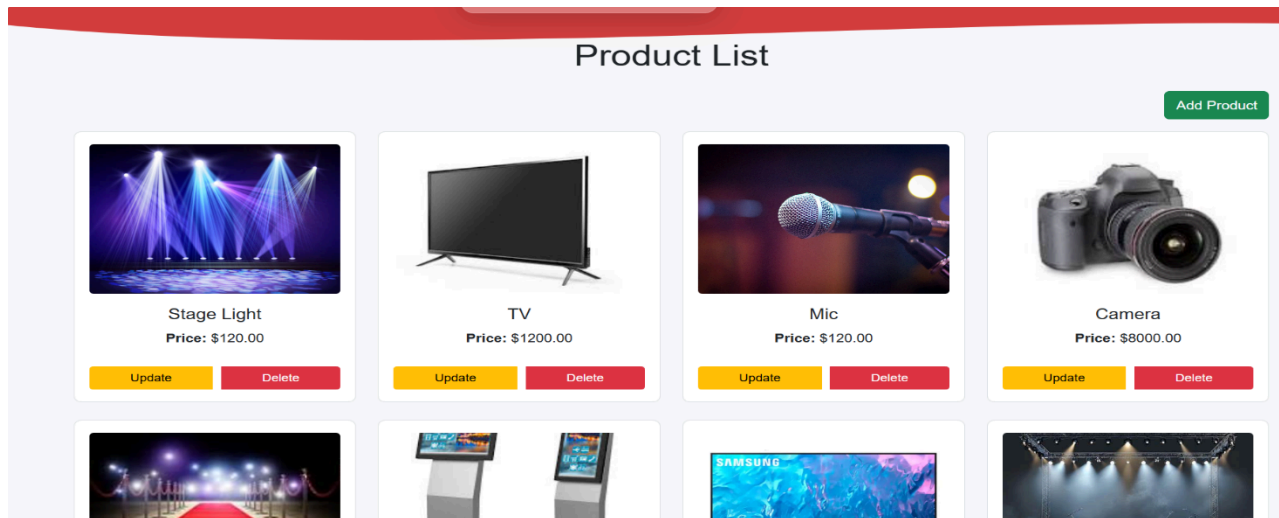


Stage Light
Price: \$120.00
[View Details](#)

- As A Seller



- Product view



- Update product

Price:

Category:

Event Product

P photo:
Currently:
[product_photos/vezels-nixabay-164829.jpg](#)

Clear

Change:

Choose File No file chosen

Status:

Stock Out

Submit

- Add product

Add Product

Name:

Description:

Price:

Category:

P photo:

No file chosen

- Seller Profile

tanisha tara

Username: Taranoon

First Name:

tanisha

Last Name:

tara

Phone:

0192

Address:

70/4

Bio:

Role

seller

You are a **Seller**. Manage your product listings below.

+ Add Product

+ Manage Products

✎ Edit Profile

Logout

Team Contributions

Tasks	22201169(Tanisha)	22201186(Sajid)	22201187(Sanjida)	22101204(Emo n)
HTML page creation	Created Profile, login, signup, and Portfolio	Created product-related HTML, Cart, and order history	Created All Meet the Team-related HTML	Create Policy, help
HTML page linking and URL mapping	Connect the profile, login signup to the homepage and their URL	Connect each HTML to the Homepage and the product	Connect each profile to the Meet the Team HTML	Connect them to the HomePage
Design with CSS	Homepage, login, signup, portfolio	Design Product list, details, cart. order history	Design Meet the team page and all profile view	Design them
Database creation in (models.py) and registration in (admin.py)	Create C-profile, Event Also, register them in admin.py	Create Product, cart, category, and order, and register them in admin.py	X	X
Backend logic handling (views.py, forms.py) and displaying data	Create a view for @loginRequire, Event-details, and event-list, handle the signup form, and display them	Create a product view for different users, product details, and a form for adding and updating products	To show the Team profile	X
Report Writing	Overall report + CEP mapping	ER diagram, alignment, and some content	X	X

Conclusion and Future Work

Conclusion

The project successfully lays the groundwork for a dual-user event management platform, demonstrating the potential to evolve into a comprehensive event planning assistant. Built using Django's MVT (Model-View-Template) architecture, the system currently supports core features including user registration and authentication, profile management, product listing and management by vendors, as well as browsing and purchasing by customers. A functional shopping cart and order processing system simulate a basic event-related marketplace, reflecting real-world workflows.

From a technical perspective, the system incorporates clean, modular architecture with scope for future expansion. The development process provided practical experience in full-stack web development with Django, encompassing user role management, form validation, dynamic URL routing, session handling, and secure file uploads. The project also demonstrated thoughtful UI design practices by integrating a modern, responsive interface.

Although core event planning tools—such as checklists, budgeting tools, and scheduling—have not yet been implemented, the current foundation is scalable and well-structured, allowing for the seamless addition of these features in subsequent phases.

Future Work

To evolve the system into a full-featured event planning assistant, several enhancements are planned:

- **Checklist System:** Users will be able to create, manage, and track task lists (e.g., "Book venue", "Send invitations"), improving event organization.
- **Budget Tracker:** A budgeting tool will allow users to plan and monitor expenses, comparing projected vs. actual costs.
- **Scheduling with Calendar Integration:** Event timelines, milestones, and automated reminders will be incorporated to help users stay on schedule.
- **Notifications:** In-app and email notifications will improve user engagement by alerting users to important updates, such as order confirmations or pending tasks.

From a user experience perspective:

- **Responsive Design Improvements:** Enhancements for mobile compatibility and accessibility will ensure a seamless experience across all devices.
- **Role-Specific Dashboards:** Customized interfaces for vendors and customers will provide a more intuitive and task-focused experience.
- **Rating and Review System:** Customers will be able to rate vendors and leave feedback, promoting trust and credibility within the platform.
- **Payment Gateway Integration:** Implementing services like SSLCommerz or Stripe will enable real-world financial transactions, making the platform deployment-ready.

By incorporating these features, the project will evolve from a basic product management platform into a user-friendly, feature-rich event planning assistant, capable of supporting users in organizing and managing memorable events with confidence and ease.

CEP Mapping

How K's are addressed throughout the project:

Ks	Attribute	How K's are addressed through the project	COs	POs
K3	Engineering fundamentals	The project utilizes web development fundamentals, including Django (MVT), authentication, and database modeling, to build the core system.	CO1	(c)
K4	Specialist knowledge	Django framework, form validation, admin panel, and file handling were applied to enable user roles, product uploads, and secure access.	CO2	(c)
K5	Engineering design	An ER diagram and modular code structure were used to design the architecture of the two-user system and future event tools.	CO2	(c)
K6	Engineering practice	HTML, CSS, Django, Bootstrap, and JavaScript were used to build practical features like product listings, carts, and checkout.	CO1, CO2	(c)
K7	Comprehension	The project promotes local vendor visibility and supports digital transformation in event planning, positively impacting society.	CO4	(f)

How P's are addressed throughout the project:

Ps	Attribute	How P's are addressed through the project	COs	POs
P1	Depth of Knowledge Requirement	It required an understanding of frontend/backend development, database handling, user sessions, and form processing in Django.	CO1	(c)
P2	Depth of analysis required	User roles, data flows (products, cart, orders), and conditional logic (e.g., form validation and access control) were analyzed in depth.	CO2	(c)
P6	Extent of Stakeholders	Vendors and customers are both essential stakeholders. The platform is designed to benefit both by streamlining communication and commerce.	CO6	(i)
P7	Interdependence	The system involves interconnected modules: authentication, profile management, product system, cart/order logic, and templates.	CO6, CO7	(i), (j)

How A's are addressed throughout the project:

As	Attribute	How are A's addressed through the project
A1	Range of Resources	Used web development tools (HTML, CSS, Django, Bootstrap), Python libraries, and real-world knowledge of event organization.
A4	Consequences for society and the environment	Encourages digitization of event planning, reduces paper use via digital checklists/reminders, and supports local vendors.